

Leçon : les bases de données

En Première, nous avons appris à manipuler des données stockées dans des fichiers CSV. Cette solution est adaptée pour de petits volumes de données. En revanche, lorsque les données deviennent nombreuses, qu'elles sont partagées par plusieurs utilisateurs ou qu'elles doivent être reliées entre elles, les fichiers CSV montrent leurs limites. Les bases de données relationnelles ont été conçues pour répondre à ces besoins.

Une **base de données relationnelle** est un ensemble organisé de données stockées sous forme de **tables reliées entre elles** par des relations.

Cette organisation est décrite par le modèle relationnel d'où l'importance de bien définir ce modèle.

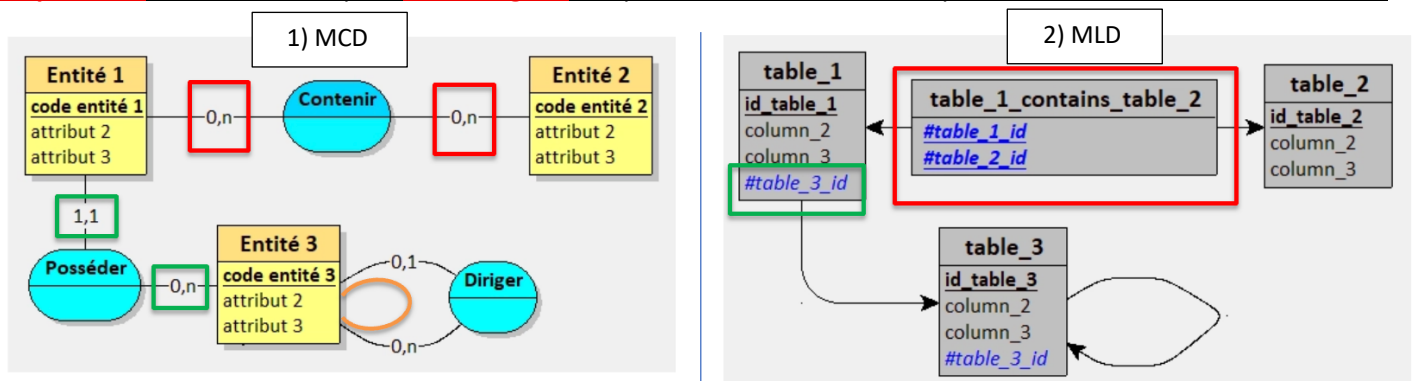
Les **requêtes SQL** sont des instructions qui permettent de consulter, ajouter ou mettre à jour ces données dans la base.

Le modèle relationnel : Il sert de support à la création des tables dans la BDR.

1^{ère} étape : construire le MCD (Modèle conceptuel de données) qui représente les entités (objets ou tables) avec leurs attributs et leurs associations (ou relations) → entité = **table** et attribut = **colonne** ou **champ**

2^{ème} étape : passer au MLD (Modèle logique de données) en ajoutant les clés primaires et les clés étrangères.

Clé primaire = identifiant unique / **clé étrangère** : clé primaire d'une autre table pour faire le lien entre les deux tables



□ association multiple $x,n \dots y,n$ devient une **table** dont la **clé primaire** combine les **identifiants** des entités impliquées.

□ association $1..n$ (ou $0..n$) ... $1,1$ (ou $0,1$) donne une **clé étrangère** à la table concernée par la cardinalité $1,1$ ou $0,1$

#clé étrangère :

- c'est la clé primaire de la table concernée par la forte cardinalité
- s'il y a des attributs dans la relation, ils basculent dans la table forte

○ **relation réflexive** : relation qui relie une entité à elle-même

- **1,N** → on ajoute une **clé étrangère** dans la **même table**.
- **N,N** → on crée une **table d'association** contenant **deux clés primaires** vers cette même table.

Un MRD mal conçu peut entraîner des redondances de données (elles sont stockées inutilement plusieurs fois), ce qui peut entraîner des anomalies d'insertion, de suppression et de mise à jour.

3^{ème} étape : représentation textuelle et linéaire pour passer au MRD (modèle relationnel de données) en ajoutant les types de données.

3) MRD

AUTEURS(id Int, nom Text, prenom Text, ann_naissance Int, langue_ecriture Text)

LIVRES(id Int, titre Text, #id_auteur Int, ann_publi Int, note Int)

Les **attributs soulignés** sont des **clefs primaires**, le # ou * signifie que l'on a une **clef étrangère**.

SGBD

Un **SGBD** (systèmes de gestion de bases de données) est le logiciel qui permet de gérer une **BD** (base de données) à l'aide de commandes SQL.

Il **s'appuie sur le modèle relationnel** pour créer une base de données et ses tables, **ajouter, modifier** et **supprimer** des données, **rechercher** des informations grâce au langage SQL, **garantir l'intégrité des données** (éviter les incohérences), **sécuriser les données** (Gérer les accès et les droits des utilisateurs, sauvegarder et restaurer les données en cas de problème).

Exemples de SGBD relationnel connus :

- **Oracle Database** → Sous Unix, robuste/performant et nécessite de connaître **PL/SQL**
- **Microsoft SQL Server** → Sous Windows avec l'outil **SQL Server Management Studio**
- **MySQL** (logiciel libre) utilisé surtout pour les sites Web
- **SQLite** (logiciel libre) → utilisé dans les applications, objets connectés...

Les **contraintes d'intégrité** sont des règles qui garantissent la cohérence et la validité. Elles empêchent d'enregistrer ou de modifier des données incohérentes ou contradictoires dans une BDR. Après la déclaration des contraintes, le SGBD rejettera toute modification des données qui ne respecte pas ces règles et risquerait ainsi de compromettre l'intégrité de la base.

Les **contraintes de domaine** restreignent les valeurs d'un attribut. Les **contraintes de référence** assurent qu'une valeur de clé étrangère existe bien en tant que valeur de clé primaire dans une autre table.

Les **contraintes de clé** garantissent que, dans une table, chaque ligne est identifiée de façon unique par la clé primaire. Les **contraintes utilisateurs** sont des règles propres à l'application (âge positif...etc).

Les **contraintes sont mises en œuvre grâce à ces mots-clefs SQL**

- **PRIMARY KEY** : clé primaire (unique et non nulle)
- **FOREIGN KEY** : clé étrangère (contrainte de référence)
- **NOT NULL** : valeur obligatoire (contrainte de domaine)
- **UNIQUE** : valeurs toutes différentes (contrainte de domaine)
- **CHECK** : condition à respecter (contrainte utilisateur)
- **DEFAULT** : valeur par défaut

Exemple de contrainte de domaine. On vérifie que les valeurs sont du bon type et dans les limites autorisées.

Ici, Impossible d'enregistrer une moyenne de 25.

```
CREATE TABLE Eleve (  
    moyenne REAL CHECK (moyenne BETWEEN 0 AND 20)  
);
```

→ Nous utilisons **SQLite** et le module **sqlite3** de **Python** en NSI.

Contrairement aux autres, il ne fonctionne **pas comme un serveur**. Toute la base de données est stockée dans un **simple fichier .db**. Le modèle relationnel sert de support à la création des tables dans le fichier **.db** du SGBD.

Un fichier **.db** est le fichier qui représente la base de données. Il contient à la fois la description des tables (le schéma de la base) et les données qu'elles stockent.

Possibilité d'ouvrir ce fichier l'outil en ligne : <https://sqliteonline.com> ou DB Browser for SQLite ou DBeaver.

Exemple d'utilisation en Python

```
import sqlite3  
  
# Connexion à la base de données  
conn = sqlite3.connect("ecole.db")  
  
# Création du curseur  
curseur = conn.cursor()  
  
# Exécution d'une requête  
curseur.execute("SELECT * FROM Eleve")  
  
# Récupération de tous les résultats  
resultats = curseur.fetchall()  
  
# Affichage des résultats  
for ligne in resultats:  
    print(ligne)  
  
# Fermeture de la connexion  
conn.close()
```

Requête SQL

Si la table Eleve contient :

id	nom	moyenne
1	Alice	15
2	Bob	12
3	Chloé	18

L'affichage sera :

```
<> Python  
  
(1, 'Alice', 15)  
(2, 'Bob', 12)  
(3, 'Chloé', 18)
```

La connexion ouvre la base de données et le **curseur** permet d'écrire des requêtes et de lire les réponses.

Les principales méthodes du curseur :

execute(requete) : exécute une requête SQL

fetchone() : retourne la première ligne du résultat (→ tuple)

fetchall() : retourne toutes les lignes

fetchmany(n) : Retourne les n premières lignes

fetchall() / fetchmany(n) retournent plusieurs lignes → liste de tuples

id	nom	moyenne
1	Alice	15
2	Bob	12
3	Chloé	18
4	David	14

UPDATE

id	nom	moyenne
1	Alice	16
2	Bob	12
3	Chloé	18
4	David	14

Modifier des données avec la commande UPDATE

```
curseur.execute(  
    "UPDATE Eleve SET moyenne = ? WHERE nom = ?",  
    (16, "Alice")  
)  
  
conn.commit()
```

id	nom	moyenne
1	Alice	16
2	Bob	12
3	Chloé	18
4	David	14

DELETE

id	nom	moyenne
1	Alice	16
3	Chloé	18
4	David	14

Supprimer des données avec la commande DELETE

```
curseur.execute(  
    "DELETE FROM Eleve WHERE nom = ?",  
    ("Bob",)  
)  
  
conn.commit()
```

Le **commit()** est indispensable après une modification (**INSERT**, **UPDATE**, **DELETE**), sinon les changements ne sont pas sauvegardés.

id	nom	moyenne
1	Alice	15
2	Bob	12
3	Chloé	18
4	David	14

INSERT

Ajouter des données avec la commande INSERT

```
curseur.execute(  
    "INSERT INTO Eleve VALUES (?, ?, ?)",  
    (4, "David", 14)  
)  
  
conn.commit() # Enregistre les modifications
```

MEMENTO SQL

Supprimer une base ou une table.

Drop table est parfois utilisé avant la création de la table pour être sûr de ne pas créer une table qui existe déjà.

- Une base :

Requête SQL

```
DROP DATABASE ma_base;
```

- Une table :

Requête SQL

```
DROP TABLE ma_table;
```

Requête SQL

```
SELECT attribut_1, attribut_3
FROM ma_table;
```

Résultat :

	titre
1	Les Aventures de Huckleberry Finn
2	Fondation et Empire
3	Akira
4	Les Robots
5	Astérix chez les Pictes
6	Les Monades urbaines

Result: 128 enregistrements ramenés en 16ms
At line 1:
SELECT titre FROM livre ;

Commande :

```
SELECT titre
FROM livre ;
```

Traduction :

On veut tous les titres de la table «livre».

- La base n'existe pas : **Créer une base de données**

Requête SQL

```
CREATE DATABASE ma_base;
```

- Au cas où la base existe et que l'on ne souhaite pas l'écraser :

Requête SQL

```
CREATE DATABASE IF NOT EXISTS ma_base;
```

```
SELECT titre
FROM livre
WHERE annee >= 1990;
```

On veut les titres de la table «livre» qui sont parus après (ou en) 1990;

Résultat :

	titre
1	Les Aventures de Huckleberry Finn
2	Fondation et Empire
3	Akira
4	Les Robots
5	Astérix chez les Pictes
6	Les Monades urbaines

Un **alias** est un nom temporaire que l'on donne à une colonne ou à une table dans une requête SQL, afin de simplifier l'écriture, de rendre le résultat plus lisible ou d'abréger des noms trop longs.

L'alias ne modifie pas la BD : il n'existe que pendant l'exécution de la requête.

```
SELECT titre AS titre_du_livre
FROM livre
WHERE annee >= 1990;
```

Lors de l'affichage du résultat et dans la suite de la requête (important), la colonne "titre" est renommée "titre_du_livre".

Résultat :

	titre_du_livre
1	Les Aventures de Huckleberry Finn
2	Fondation et Empire
3	Akira
4	Les Robots
5	Astérix chez les Pictes

SQL fonctions d'agrégat

```
SELECT FONCTION(colonne1)
FROM matable;
```

Voici les **fonctions d'agrégat** les plus courantes :

- **COUNT** : compte le nombre de lignes ou d'éléments (ex : nombre d'élèves)
- **SUM** : calcule la somme d'une colonne numérique (ex : total des notes)
- **AVG** : calcule la moyenne (ex : moyenne des âges)
- **MIN** : donne la plus petite valeur (ex : âge du plus jeune élève)
- **MAX** : donne la plus grande valeur (ex : note maximale)

```
SELECT COUNT(*) AS total
FROM livre
WHERE titre LIKE "%Astérix%";
```

On veut compter le nombre d'enregistrements de la table livre comportant le mot "Asterix". Le résultat sera le seul élément d'une colonne nommée "total".

Résultat :

	total
1	10

```
SQL
SELECT colonnes
FROM matable
ORDER BY colonne1 [ASC|DESC], colonne2... ;
```

Pour organiser les résultats d'une requête, on va utiliser la clause ORDER BY.

Dans la clause **ORDER BY**, on spécifie :
→ **ASC** pour organiser les résultats par ordre croissant (mode par défaut) ou
→ **DESC** pour les trier par ordre décroissant.

Créer les tables.

Exemple de script avec une seule clé primaire :

Requête SQL

```
CREATE TABLE ma_table (
  attribut_1 INTEGER PRIMARY KEY AUTOINCREMENT,
  attribut_2 TEXT,
  attribut_3 VARCHAR(50)
);
```

- Clé primaire multiple :

Requête SQL

```
CREATE TABLE ma_table (
  attribut_1 INTEGER,
  attribut_2 TEXT,
  attribut_3 VARCHAR(50),
  PRIMARY KEY (attribut_1, attribut_2)
);
```

- Clé étrangère :

Requête SQL

```
CREATE TABLE ma_table (
  attribut_1 INTEGER PRIMARY KEY AUTOINCREMENT,
  attribut_2 INTEGER,
  attribut_3 VARCHAR(50),
  FOREIGN KEY (attribut_2) REFERENCES autre_table (attribut_de_reference)
);
```

→ **Sélectionner** le contenu d'une table avec **SELECT**

Requête SQL

```
SELECT *
FROM ma_table;
```

SELECT * FROM livre

On obtient alors l'intégralité de la table livre.
L'astérisque * indique que l'on affiche **toutes** les colonnes.

Structure générale d'une requête SELECT

SQL

```
SELECT [DISTINCT] colonne1, colonne2, ...  
FROM matable  
[WHERE condition]  
[ORDER BY tri]  
[LIMIT nombre];
```

Remarque

Les crochets [] signifient que les parties sont optionnelles.

Structure générale d'une requête SELECT avec JOIN

Une **jointure** permet de lier deux tables dans une BD, grâce à un ou plusieurs champs qu'elles ont en commun (généralement une clé étrangère et une clé primaire).

Cela permet de consulter des données « complètes » en une seule requête.

```
SELECT ...  
FROM ...  
JOIN ...  
ON ...  
WHERE ...  
ORDER BY ...  
LIMIT ...
```

```
SELECT livre.titre, emprunt.code_barre, emprunt.retour  
FROM emprunt  
JOIN livre ON emprunt.isbn = livre.isbn;
```

```
SELECT u.nom, u.prenom, l.titre, e.retour  
FROM emprunt AS e  
JOIN livre AS l ON e.isbn = l.isbn  
JOIN usager AS u ON e.code_barre = u.code_barre;
```

Commande :

```
SELECT titre, annee  
FROM livre  
WHERE titre LIKE "%Astérix%"  
ORDER BY annee DESC;
```

Traduction :

On veut afficher tous les albums d'Astérix, et leur année de parution, classés par année décroissante.

- **Comportement par défaut :** Si le paramètre ASC ou DESC est omis, le classement se fait par ordre **croissant** (donc ASC est le paramètre par défaut).

- **Utilisation de LIMIT :** Le mot-clé LIMIT (suivi d'un nombre) permet de limiter le nombre de résultats affichés. Ainsi la requête

```
SELECT titre, annee  
FROM livre  
WHERE titre LIKE "%Astérix%"  
ORDER BY annee DESC  
LIMIT 1;
```

permet d'obtenir les renseignements sur l'Astérix le plus récent.

SELECT avec LIMIT pour n'afficher que le nombre demandé de lignes (ici un seul résultat)

Commande :

```
SELECT DISTINCT editeur  
FROM livre;
```

Traduction :

On veut la liste de tous les éditeurs. Sans le mot-clé **DISTINCT**, beaucoup de doublons apparaîtraient.

Résultat :

	editeur
1	Flammarion
2	Editions Denoël
3	Glénat
4	Editions Milan

SELECT avec DISTINCT

Même si la définition des clés dans la base de données empêche la présence de doublons dans les tables, cela n'empêche pas que le résultat d'une requête puisse contenir, lui, des doublons.

Dans une base de données relationnelle, **les requêtes de modification permettent d'ajouter, de modifier ou de supprimer des données contenues dans les tables.**

Ces requêtes sont essentielles pour faire évoluer le contenu d'une base au fil du temps.

Il y a 3 types de requêtes de modification en SQL :

INSERT : ajouter des lignes (ou enregistrements) dans une table

UPDATE : modifier des valeurs d'une ou plusieurs colonnes ou lignes

DELETE : supprimer des lignes de la table.

Pour modifier la note d'Alice utiliser UPDATE :

```
UPDATE Table_notes  
SET Maths = 18  
WHERE Nom = 'Alice';
```

Pour supprimer, totalement la ligne contenant Charles, utiliser DELETE :

```
DELETE FROM Table_notes  
WHERE Nom = 'Charles';
```

Pour ajouter 3 lignes dans la table Table_notes, utiliser INSERT :

id	Nom	Maths	Anglais	NSI
1	Alice	16	11	17
2	Bob	12	15	10
3	Charles	9	11	18

```
INSERT INTO Table_notes  
VALUES (1, 'Alice', 16, 11, 17),  
       (2, 'Bob', 12, 15, 10),  
       (3, 'Charles', 9, 11, 18);
```

Essayons d'insérer un 4ème enregistrement ayant le même id qu'un autre élève :

```
INSERT INTO Table_notes  
VALUES (3, 'Denis', 18, 10, 12);
```

La contrainte de relation est violée : le SGBD protège la BD en n'acceptant pas la proposition d'insertion. La BD n'est pas modifiée.

```
UNIQUE constraint failed: Table_notes.Id: INSERT INTO Table_notes VALUES (3, 'Denis', 18, 10, 12);
```

Il est possible d'utiliser l'instruction **AUTOINCREMENT** lors de la déclaration de la table afin que l'attribut id soit géré automatiquement par le SGBD.

```
CREATE TABLE Table_notes (  
  Id INTEGER PRIMARY KEY AUTOINCREMENT,  
  Nom TEXT,  
  Maths INTEGER,  
  Anglais INTEGER,  
  NSI INTEGER  
);
```

```
INSERT INTO Table_notes  
VALUES  
  ('Alice', 16, 11, 17),  
  ('Bob', 12, 15, 10),  
  ('Charles', 9, 11, 18);
```